Uptime Cairn

An all-in-one open source uptime monitoring
& reporting platform

> *A cairn is a stack of stones built up by many passing travellers to mark the safe path for whoever comes next. Stacked stones also happen to look exactly like an uptime bar.*

Document Project Plan, Draft v1

Date 18 July 2026

Home uptimecairn.dev

Licence AGPLv3 with CLA

0. The Objective

Uptime Cairn is **one** open source uptime monitoring and reporting tool that serves the entire spectrum of users — a freelancer with three client websites, a 10-person startup, an MSP managing 200 customers, and an enterprise with SOC 2 auditors — without artificial feature gating, without a paid tier holding the good parts hostage, and without forcing anyone to bolt together four separate tools.

This is **not** a plan optimised for extracting revenue at every layer. It is a plan optimised for one tool being genuinely sufficient for everyone.

That objective has a direct architectural consequence, and it is the single most important idea in this document:

> **One codebase. One binary. Progressive disclosure.**
>
> The freelancer never sees RBAC, escalation policies, or Terraform. The enterprise admin never has to migrate to a different product to get them. The same install grows with the user. Complexity is revealed on demand, never imposed by default.

Every decision below is downstream of that.

---

1. Why This Should Exist — Evidence

1.1 The category leader has a well-documented ceiling

Uptime Kuma is the community standard — 88k+ GitHub stars, ~40 monitor types, ~95 alert integrations, one Docker container. It is genuinely excellent at what it does. But the gap between what it provides and what teams need is documented and widening. Recurring findings across independent sources:

Missing capability	Consequence
No write REST API	All mutations require a Socket.IO client — a hard barrier for CI/CD and automation
No RBAC / multi-user	One admin account with full control; anyone with dashboard access can delete anything
No distributed probing	Monitors from a single location; if that server dies, alerting dies with it
No SSO	Blocks any organisation with an identity provider
SQLite-bound	Scaling ceiling on retention and monitor count
No on-call scheduling	Alerts fire immediately, always; no rotation, no business-hours suppression

Missing capability	Consequence
No incident management	Incidents are posted **by hand**, not opened automatically from a failing check
No config-as-code	No Terraform, no YAML apply, no GitOps
Weak historical reporting	Default graphs show roughly a week; long-term history stored but not browsable
Hard scale wall at ~300–600 monitors	Socket.IO pushes full state to every browser client; UI becomes unusable. See §1.4(a) — this is the most severe limitation of all
No webhook payload templating	Outgoing webhooks can't be shaped to fit the receiving system
No HA	Confirmed unavailable and not planned upstream; the Helm chart is single-pod

At the time of writing the upstream project carries several hundred open issues, many of them highly-upvoted requests for exactly the items above.

1.2 The workaround ecosystem is visibly failing

The clearest signal in the entire research set. Two community projects exist *solely* because there is no official write API — and both are collapsing under that dependency:

`lucasheld/uptime-kuma-api` (392★, 23 open issues) - Open issue #75: "*This project seems to be abandoned*" — opened Sep 2024, still open - Open issue #85: "*Update to match Uptime Kuma 2.x*" — the wrapper cannot keep pace - Open issue #91: "*delete_monitor() function doesn't work*" - Open issue #73: "*Add support for API Key instead of Username/Pass authentication?*"

`MedAziz11/Uptime-Kuma-Web-API` (277★, 39 open issues) - Open issue #89: "*Project maintenance status inquiry*" - Open issue #88: "*High memory consumption*" - Open issue #85: "*Get Monitors does not include a schema*" - Open issue #75: "*/statuspages runs into Internal Server Errors*"

The upstream feature request for a native write API states the root cause plainly: community wrappers depend on Socket.IO internals and break with upstream updates.

Read this correctly. This is not "someone should build a better wrapper." This is proof that a monitoring tool whose API is an afterthought will spawn a fragile, unmaintained satellite ecosystem that eventually strands its users. It is the strongest possible argument for API-first design.

1.3 The alternatives each solve one slice

- **OpenStatus** — excellent platform engineering: 28 regions across 3 clouds, YAML + CI/CD monitor versioning, Terraform provider, MCP server, private probe container, 8.5MB image. But narrow protocol coverage (HTTP endpoints, REST/GraphQL), and its free tier is 1 monitor / 1 status page / 10-minute minimum interval.
- **Gatus** — highly configurable conditions, but YAML-only setup is a barrier and the UI is comparatively spare.

- **Kener** — best-looking status page of the group, custom RBAC roles, 24 locales, embeddable widgets. But no multi-region probing, four alert channels, single tenant, hard Redis dependency.
- **Checkmk / Zabbix** — enterprise-capable and scalable, but heavyweight, steep learning curve, wrong shape entirely for a freelancer.
- **Better Stack / Pingdom / UptimeRobot** — polished and complete, but proprietary, per-monitor pricing, and your data lives on someone else's infrastructure.

No project occupies the square: broad protocol coverage + modern platform engineering + team/enterprise controls + serious reporting, in one open source install. That square is the whole opportunity.

1.4 Primary source: what the r/selfhosted community actually said

The r/selfhosted thread was read directly (via screenshots — Reddit hard-blocks automated fetchers). It is the most valuable source in this document, because it is users describing their own pain rather than vendors describing a market. Five findings, in order of importance:

(a) The hard scale ceiling — the single most important finding in this plan.

One user: they loved Kuma, it was perfect — *until they crossed roughly 300 monitors at once*. They have nearly 1000 sites to monitor and Kuma cannot handle it. A reply confirms independently: loaded 600 URLs, "*I can barely use the app. It's just stuck on websocket errors at this point.*" The community's only suggested workaround is "*fork another instance of kuma on another host.*"

This is decisive. It is a **hard, reproducible, numeric wall at ~300–600 monitors**, caused by the Socket.IO architecture pushing full state to every connected browser client. It cannot be tuned around; it is structural. And note precisely who hits it: someone with 1000 client sites — **exactly the agency/MSP persona identified in §3 as the strategic wedge.** The most underserved segment is being ejected from the category leader by an architectural limit, and their only option is to shard by hand across hosts.

This gives Uptime Cairn an unambiguous, testable, headline claim: *5,000 monitors on one install, and the UI stays fast.* No competitor in the open source space can say that today.

(b) The API gap, confirmed from the user side.

Top-level comment, +13: "*Does anybody have a good way to add/remove monitors via api/curl? I know there's a discussion on adding a usable api, but that hasn't gone anywhere. I bodged together a json generator to add a bunch of sites, but it's a total one-off and not pretty.*" The only reply points at the two wrapper repos from §1.2 — both of which are now failing. This is the exact demand → broken-workaround → abandonment chain, observed end to end.

(c) Webhook payloads cannot be customised. +12: "*The only thing I don't like is that I can't do custom stuff with webhooks.*" Small feature, disproportionate frustration. Users want to shape the outgoing payload to fit the system receiving it. **Cheap to build, high goodwill, add to Phase 1.**

(d) HA is asked for and explicitly unavailable. A user asks whether an HA Kuma setup is possible. The Kubernetes/Helm suggestion is corrected: "*This deployment isn't HA. It's self healing but there's still only one pod.*" Final answer: "*There is no way at this time and from the discussions on the issues/PRs there doesn't seem like one will be coming any time soon.*" Confirms HA as unmet, and confirms it is not arriving upstream.

(e) Everything else is either too little or too much. Prometheus blackbox_exporter is recommended but conceded to be "*a bit of an overshoot if you use that toolstack just for online monitoring.*" Zabbix: "*this would be overkill for just uptime monitoring.*" ELK + Heartbeat: "*not the easiest but the most complete product for me.*" This is the positioning gap stated by users in their own words — the middle ground between a toy and an observability platform is empty.

Two secondary notes. A user raises "*pervasive supply-chain risks in npm*" as an unacknowledged concern — relevant to §5.5's language choice. And a user on free-tier container hosting found their instance "*can't ping any IP's (even google)*" — a containerised-ICMP failure mode worth handling explicitly with a clear error message and a documented fallback to TCP checks.

Discounted: several comments in the thread are self-promotional posts from a single vendor account, posted years after the surrounding discussion. Not treated as signal.

---

2. Product Principles

Non-negotiable. Every proposed feature is tested against these.

1. **Sixty seconds to first monitor.** `docker run` → open browser → monitor running. If onboarding is harder than Uptime Kuma's, adoption dies no matter how good the rest is.
2. **No feature is paywalled in the open source build.** RBAC, SSO, multi-region, PDF reports, audit logs — all of it ships in the AGPL build. Uptime Cairn earns money from hosting and support, never by crippling the software.
3. **API-first, literally.** The full surface is specified in OpenAPI/protobuf *before* the UI exists. The dashboard is the first API client, not a privileged one. This is the direct lesson of §1.2.
4. **Progressive disclosure.** Advanced surfaces stay hidden until the user opts in. A solo user must never see an "escalation policy" field.
5. **Sensible defaults, deep overrides.** It should work well with zero configuration and bend fully when configured.
6. **Reporting is a first-class product, not a graph.** See §4 — this is the most under-served need in the whole category.
7. **Your data is yours.** Full export in open formats, no lock-in, no phone-home, telemetry opt-in only.
8. **Never lose a heartbeat.** Monitoring you can't trust is worse than none.
9. **The UI must stay fast at 5,000 monitors.** Paginated, filtered, server-side queries; the client is never sent the full state. This is a hard requirement from day one, not an optimisation — it is the specific failure that ejects agencies from Uptime Kuma (§1.4a). Load-test it every release.
10. **Minimal dependency surface.** Few third-party packages, vendored and pinned, SBOM published, reproducible builds. Users self-hosting infrastructure monitoring are justifiably wary of transitive supply-chain risk.

---

3. Users and What Each Needs

Persona	Scale	What they need	What must stay invisible
Freelancer / solo dev	5–30 monitors	One-command install, simple UI, email/ Telegram alerts, a shareable status page, **branded monthly client reports**	RBAC, on-call, regions, config-as-code
Agency / MSP	100–2000 monitors, many clients	**Multi-tenancy with client isolation**, white-label per client, per-client status pages, automated PDF reports, bulk operations	Enterprise SSO complexity
Startup team	20–200 monitors	On-call rotation, escalation, Slack/PagerDuty, incident timeline, config-as-code	Multi-tenancy, compliance exports
Enterprise	1000+ monitors	SSO/SAML, RBAC, audit log, HA, multi-region, private probes, SLO/error budgets, compliance-grade reports	— (needs everything)
Homelabber	3–50 monitors	Tiny footprint, SQLite mode, runs on a Pi, no external dependencies	Essentially everything above

The agency/MSP persona is the strategic wedge. They are chronically underserved — they need multi-tenancy and white-label reporting, which no open source option provides properly, and commercial tools price per monitor, which is brutal at their volume. They are also loud, well-networked, and they *bring their clients with them*.

---

4. The Differentiator: Reporting

The objective explicitly says "monitoring **and reporting**." Take that literally, because it is where the entire category is weakest. Uptime Kuma's historical UI shows roughly a week by default; long-term data is stored but not browsable. Everyone else treats reporting as a dashboard screenshot.

Reporting engine — a first-class subsystem, not a graphs page:

- **Scheduled reports** — daily / weekly / monthly / quarterly, auto-generated and auto-delivered via email, Slack, webhook, or S3 drop.
- **White-label / branded** — the agency uploads a logo, sets colours and a footer, and the client receives a report that looks like the agency made it. This alone will win the MSP segment.
- **Multiple output formats** — PDF, HTML, CSV, JSON, and a public shareable link.
- **SLA / SLO reports** — target vs. actual, error budget consumed and remaining, burn rate, breach log with timestamps. This is what auditors and contract reviews need.
- **Incident post-mortem reports** — auto-drafted from the incident timeline: detection time, acknowledgement, resolution, MTTD/MTTA/MTTR, affected components, alerts fired.
- **Comparative reporting** — period over period, monitor vs. monitor, region vs. region.

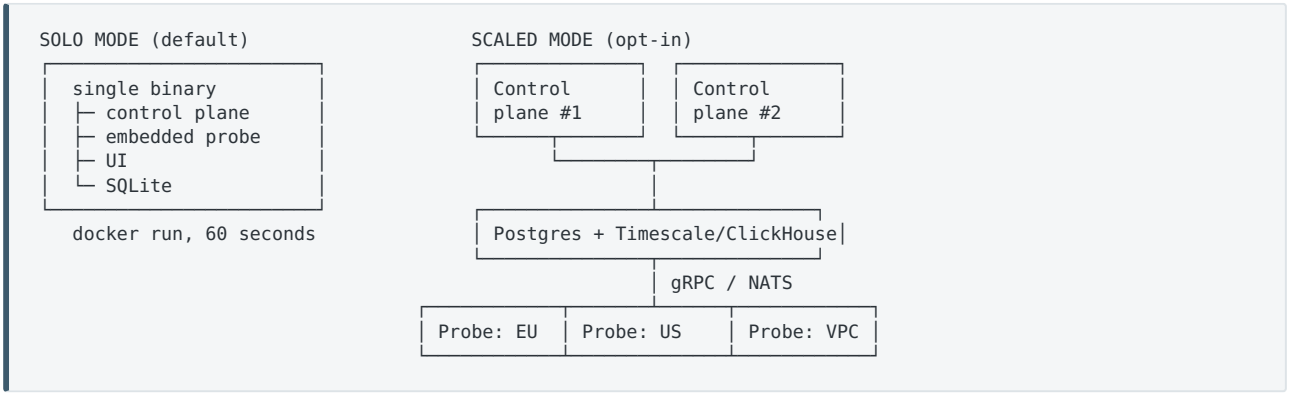
- **Custom report builder** — pick metrics, group by tag/client/region, choose a window, save as a reusable template.
- **Full historical browsing** — retention limited only by disk, with real drilldown into arbitrary past ranges. Fix the thing everyone else got wrong.
- **Certificate & domain expiry reports** — a forward-looking calendar, not just an alert three days before it breaks.

Reporting is the feature most likely to make someone switch, because it is the feature they currently produce by hand in a spreadsheet every month.

---

5. Architecture

5.1 Two deployment shapes, one codebase



Same binary, different flags. The upgrade path from solo to scaled is a config change and a database migration — **never a reinstall, never a different product.**

5.2 Control plane / probe split

The single most consequential decision, and it must be made in week one because it cannot be retrofitted. Probes are stateless agents that register with the control plane over gRPC (or NATS for fan-out) and pull their assignments.

This one split unlocks, simultaneously: - Multi-region monitoring - Private probes behind a firewall or inside a VPC (agent dials out; no inbound ports) - Horizontal scale — add probes, add capacity - HA — control plane failure doesn't stop checks; probes buffer and replay - Consensus checking — require *N of M* regions to agree before declaring an outage, which kills the single biggest source of false-positive pages

In solo mode the probe is simply compiled in and runs in-process. The user never knows the split exists.

5.3 Storage

- **Solo:** embedded SQLite, zero dependencies, runs on a Raspberry Pi.
- **Scaled:** PostgreSQL for configuration and relational state; **TimescaleDB or ClickHouse** for heartbeat time-series.

- Automatic tiered rollups: raw → 1-minute → 5-minute → hourly → daily. Keeps years of history queryable without unbounded disk growth. This is what makes §4's long-range reporting actually possible.
- Optional Redis for caching and pub/sub — **optional**, never required. (Kener's hard Redis dependency is a documented adoption barrier; do not repeat it.)

5.4 API-first contract

- Full CRUD over **every** entity: monitors, tags, groups, notifications, status pages, incidents, maintenance windows, users, teams, schedules, reports.
- REST + OpenAPI spec, generated clients, versioned (`/api/v1`) with an explicit deprecation policy.
- Scoped API keys — per-token permissions, expiry, last-used tracking, revocation.
- Webhooks out for every state change.
- Native Prometheus `/metrics` and OpenTelemetry export.
- **Contract tests in CI.** The API is a promise; §1.2 is what happens when it isn't.

5.5 Suggested stack

Layer	Choice	Why
Backend	**Go**	Single static binary, low memory, excellent concurrency for thousands of parallel checks, trivial cross-compilation for ARM/Pi
Probe	Go	Same binary, `--mode=probe`
Frontend	SvelteKit or React + Tailwind + shadcn/ui	Fast, small bundle, good component ecosystem
DB	SQLite / Postgres + Timescale	Covers both ends of the range
Transport	gRPC + Protobuf, NATS optional	Typed, efficient, streaming
Browser checks	Playwright in a sidecar container	Only pulled when the user enables it
Reports	Typst or headless Chromium → PDF	Typst is far lighter if the templating suffices

Go over Node is deliberate: memory footprint matters at both extremes — the Pi user and the 5000-monitor enterprise. Recall that one of the wrapper projects has an open issue titled "*High memory consumption.*"

---

6. Roadmap

Phase 0 — Foundations (weeks 1–4)

Repository, licence, CoC, governance doc, CI/CD. Data model. **API spec written and frozen before code.** Probe protocol design. Architecture Decision Records from day one.

Exit: an ADR set and an OpenAPI spec a stranger could implement against.

Phase 1 — Solid Core (months 1–4) · *must be independently useful*

Monitor types: HTTP/HTTPS (status, keyword, JSON path, regex, response time threshold), TCP port, ICMP ping, DNS record, SSL/TLS expiry, domain expiry, push/heartbeat (dead-man's switch), Docker container, gRPC.

Alerting: email (SMTP), webhook, Slack, Discord, Telegram, Matrix, Gotify, ntfy, Microsoft Teams, PagerDuty, Opsgenie, Twilio/SMS — **plus Apprise as a meta-provider**, which buys ~90 additional channels for a fraction of the effort.

Core: groups, tags, dependency-aware suppression, retries, timeouts, custom intervals (down to 20s), maintenance windows, public status pages with custom domains, dark mode, i18n scaffolding, full REST API, Prometheus metrics.

Webhook templating — user-defined payload bodies, headers, and HTTP method, with variable interpolation (`{{monitor.name}}`, `{{status}}`, `{{response_time}}`, etc.) and a live preview. Directly answers §1.4(c); cheap to build, disproportionately appreciated.

Scale from day one — server-side pagination, filtering and search; the client is never sent full state. **Acceptance gate: 5,000 monitors on one install with a UI that stays responsive**, verified by automated load test in CI. This is the headline claim against the category leader and it must be true at v0.1, not promised for later.

ICMP handling — detect restricted container environments where raw sockets are unavailable, fail with a clear explanatory message rather than silent breakage, and offer an automatic fallback to TCP checks (§1.4, secondary notes).

Migration: `import kuma <path-to-kuma.db>` — reads a Kuma SQLite file and reproduces monitors, tags, notifications, and status pages. **P0, not a nice-to-have.** This is the single highest-leverage growth feature in the entire plan; every existing Kuma user is a 30-second migration away.

Exit: a solo user can replace Uptime Kuma entirely and never think about it again — **and an agency with 1,000 monitors can too**, which Uptime Kuma cannot serve at all.

Phase 2 — Reporting (months 4–7) · *the differentiator*

Everything in §4. Scheduled reports, white-label branding, PDF/HTML/CSV/JSON export, SLA and SLO tracking with error budgets, incident post-mortems, comparative reports, custom report builder, unlimited historical browsing, expiry calendars.

Ship this **before** the enterprise controls. It is what makes people talk about the project, and it is what freelancers and agencies feel immediately.

Exit: an agency can send 50 branded client reports on the 1st of every month, without touching anything.

Phase 3 — Teams & Multi-Tenancy (months 7–11)

Organisations and workspaces with hard data isolation. RBAC (owner / admin / editor / responder / viewer / billing), extensible with custom roles. Local auth + OIDC + SAML + LDAP. Full audit log — who changed what, when, from where. Per-client white-labelling.

Incident management: incidents open **automatically** from failing checks (the thing Kuma makes you do by hand), with acknowledgement, assignment, a threaded timeline, status page linkage, and auto-resolve.

On-call: schedules, rotations, overrides, holiday calendars, escalation policies, business-hours-only routing, alert deduplication and grouping, quiet hours.

Exit: a 50-person engineering org can run production on-call from this and nothing else.

Phase 4 — Scale & Platform (months 11–16)

Multi-region probes with consensus checking. Private probe agents for VPC/firewall targets. HA control plane. Postgres/Timescale path with tiered rollups. Terraform provider. YAML + CLI `apply` with a GitHub Action. MCP server so assistants can manage monitors. Browser/synthetic checks via Playwright. Multi-step API flows. SNMP, MQTT, Radius, database checks (Postgres/MySQL/Mongo/Redis). Backup/restore and disaster recovery.

Exit: feature-complete against every commercial competitor.

Phase 5 — Depth (month 16+)

Latency anomaly detection against learned baselines. Predictive expiry and capacity warnings. AI-drafted incident summaries. Plugin SDK for custom monitor types and notification channels. Mobile apps. Public template gallery for status pages and report layouts.

---

7. Governance, Licence, Sustainability

Licence: AGPLv3, with a CLA. AGPL prevents a hyperscaler strip-mining the work into a closed SaaS without contributing back. The CLA preserves the option to relicense or dual-licence later if the project's needs change. Given Uptime Cairn's "not profit at every angle" objective, AGPL is the correct instrument: it protects openness rather than monetising restriction.

Bus factor is the top existential risk. Both wrapper projects in §1.2 are dying of single-maintainer burnout, and the upstream project is famously one person. Contributors and adopters *evaluate this now.*

Mitigations, non-optional: - **3+ committers with merge rights before v1.0.** - A published `GOVERNANCE.md` with a documented succession plan. - Ruthless CI: tests, linting, contract tests,

reproducible builds. Lower the cost of contributing. - `good-first-issue` triage and a genuinely responsive review cadence. - Public roadmap and monthly progress notes.

Sustainability without compromising the mission: managed hosting for people who don't want to self-host; commercial support and SLAs for enterprises; sponsorships and paid priority features. **Never** by gating capability in the open build.

On the name. *Uptime Cairn* deliberately echoes *Uptime Kuma*. That is the point: it reads as a successor in the same lineage rather than a rival, which is exactly the posture described below. Existing Kuma users parse it instantly.

Relationship with Uptime Kuma: cooperative, always. 88k stars of accumulated goodwill. Position as "*the tool you graduate to, when you need teams and reporting*" — never as a replacement or a competitor. Credit it publicly. Ship a flawless importer. Contribute fixes upstream where they apply.

---

8. Success Metrics

Milestone	Target
Phase 1 release	500 GitHub stars, 50 self-hosted installs reporting in (opt-in)
Phase 2 release	2,000 stars, 10 agencies using white-label reports in production
Phase 3 release	5,000 stars, 3+ regular external committers, first 100-person org in production
Phase 4 release	15,000 stars, listed as a first-class alternative on major comparison sites
Health, always	Median issue first-response < 48h; no more than 2 consecutive weeks without a release

9. Risks

Risk	Severity	Mitigation
Scope collapse — this plan is roughly Kuma + PagerDuty + Statuspage + a BI tool. The normal outcome is 40% built and abandoned.	**Critical**	Phase 1 must be shippable and genuinely useful standing alone. If it can't win real users by itself, no later phase will save the project. Cut features, never phases.
Single-maintainer burnout	**Critical**	§7 governance measures. Treat recruiting committers as a P0 deliverable, not a background hope.
Competitors move first — OpenStatus is bootstrapped, profitable and shipping tooling fast	High	Their gap is protocol breadth, team controls, and reporting. Reporting (Phase 2) is defensible ground they show no sign of prioritising. Get there first.

Risk	Severity	Mitigation
Trying to serve everyone serves no one	High	Progressive disclosure is the answer, and it must be enforced in UX review, not assumed. Test the solo onboarding path every single release.
Multi-tenant data isolation bug	High	Tenant isolation tests in CI from Phase 3 day one; third-party security audit before v1.0.
Monitoring tool that itself goes down	Medium	HA control plane, probe-side buffering, self-monitoring, and a documented "who watches the watchman" pattern.
Performance regressions at scale	Medium	Continuous load testing against a 10,000-monitor synthetic workload from Phase 1 onward.

10. Immediate Next Steps

1. **Build the load-test harness before the product.** A 5,000-monitor synthetic workload and a UI responsiveness benchmark, wired into CI from the first commit. The scale claim in §1.4(a) is Uptime Cairn's central promise; it must be measured continuously, not discovered late.
2. Interview 10 people — weight heavily toward **agencies and MSPs running 300+ monitors**, since §1.4(a) shows they are actively being ejected from the incumbent and have nowhere to go. Validate the reporting thesis with them at the same time.
3. Write the ADRs: probe protocol, storage strategy, tenancy model, **and the state-synchronisation model for the UI** (the specific thing Uptime Kuma got wrong).
4. Write the OpenAPI spec. Publish it for comment **before** writing code.
5. Publish `GOVERNANCE.md` and `ROADMAP.md` under the `uptimecairn` org.
6. Register `uptimecairn.dev` (canonical), `.com`, `.net`, `.co`; claim the GitHub org `uptimecairn`, Docker Hub, the `@uptimecairn` npm scope and social handles on the same day. Note that bare *Cairn* is heavily contested in software (an AI coding agent, a Google P4 compiler, a C++20 build tool, an energy-systems optimiser); the qualified form is what's defensible. Ship the CLI as `cairn` with an `uptimecairn` alias. Run a proper trademark clearance before any spend.
7. Build the Kuma importer as a standalone tool first — it is useful on its own, validates the data model, and starts building an audience before v0.1 exists. Target the sharded-across-multiple-hosts case explicitly: **merging several Kuma instances into one install is a migration story nobody else can offer.**

Uptime Cairn — living document. Revise it as evidence arrives; do not defend it as written.